Project Planning Phase

Project Planning Template (Product Backlog, Sprint Planning, Stories, Story points)

Date	1 October 2026
Team ID	SWTID-2026-5313
Project Name	Auto Ticket Classification
Maximum Marks	5 Marks

Product Backlog, Sprint Schedule, and Estimation (4 Marks)

Use the below template to create product backlog and sprint schedule

Sprint	Functional Requirement (Epic)	User Story Number	User Story / Task	Story Points	Priority	Team Members
Sprint-1	Ticket Intake	USN-1	As a user, I can submit a support ticket with subject and description.	2	High	
Sprint-1	Ticket Intake	USN-2	As a system, I can validate and store ticket details for classification.	2	High	
Sprint-2	Text Processing	USN-3	As a system, I can preprocess ticket text before classification.	3	High	
Sprint-2	Classification	USN-4	As a system, I can predict the appropriate ticket category.	4	High	
Sprint-3	Priority & Routing	USN-5	As a system, I can predict the priority of a support ticket.	2	High	
Sprint-3	Priority & Routing	USN-6	As a system, I can route a classified ticket to the appropriate support team.	3	High	
Sprint-4	Dashboard & Evaluation	USN-7	As an admin, I can view ticket category, priority, and routing results.	2	Medium	
Sprint-4	Dashboard & Evaluation	USN-8	As an admin, I can view classification performance metrics.	2	Medium	

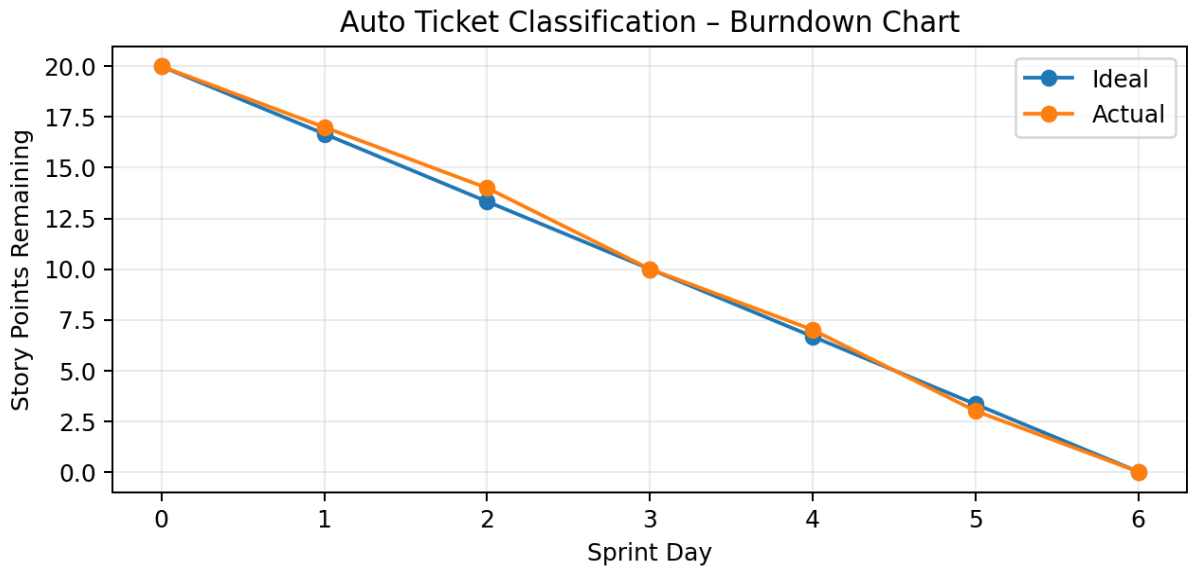
Project Tracker, Velocity & Burndown Chart: (4 Marks)

Sprint	Total Story Points	Duration	Sprint Start Date	Sprint End Date (Planned)	Story Points Completed (as on Planned End Date)	Sprint Release Date (Actual)
Sprint-1	4	6 Days	05 Oct 2026	10 Oct 2026	4	10 Oct 2026
Sprint-2	7	6 Days	12 Oct 2026	17 Oct 2026	7	17 Oct 2026
Sprint-3	5	6 Days	19 Oct 2026	24 Oct 2026	5	24 Oct 2026
Sprint-4	4	6 Days	26 Oct 2026	31 Oct 2026	4	31 Oct 2026

Velocity:

For the Auto Ticket Classification project, the planned work is 20 story points across four 6-day sprints. The average velocity is calculated as total completed story points divided by total sprint days.

Average Velocity (AV) = 20 story points ÷ 24 sprint-days = **0.83 story points/day**



Burndown Chart:

A burndown chart is a graphical representation of work left to do versus time. It is often used in Agile software development methodologies such as Scrum. For this project, the chart tracks the remaining story points as the Auto Ticket Classification features are completed.

Project-specific interpretation: The sprint starts with 20 story points. The remaining work reduces toward 0 as ticket intake, text processing, classification, priority/routing, and dashboard/evaluation tasks are completed.

Reference:

The Agile/Scrum planning structure in this document follows the supplied project planning template and its listed references.

https://www.atlassian.com/agile/project-management

https://www.atlassian.com/agile/tutorials/how-to-do-scrum-with-jira-software

https://www.atlassian.com/agile/tutorials/epics

https://www.atlassian.com/agile/tutorials/sprints

https://www.atlassian.com/agile/project-management/estimation

https://www.atlassian.com/agile/tutorials/burndown-charts

FSD Documentation – Auto Ticket Classification
Team ID: SWTID-2026-5313

1. Introduction

Project Title: Auto Ticket Classification
Team Members:
R Priyadharshini
K Pradeeba
Keerthika
Ponselvi
Deepika

2. Project Overview

Purpose: Automate classification of support tickets using ML and NLP.
Features: Automatic categorization, real-time classification, user-friendly UI, scalable system.

3. Architecture

Frontend: React.js
Backend: Node.js & Express.js
Database: MongoDB

4. Setup Instructions

Prerequisites: Node.js, MongoDB, Python
Installation: Clone repo, install dependencies, set environment variables, run servers.

5. Folder Structure

Client: React frontend
Server: Backend APIs and ML logic

6. Running the Application

Frontend: npm start (client)
Backend: npm start (server)

7. API Documentation

POST /classify → Input: Ticket text, Output: Category
GET /tickets → Fetch tickets

8. Authentication

JWT-based authentication with secure login and access control

9. User Interface

Ticket input form, classification result display, dashboard

10. Testing

Unit testing, model accuracy testing, UI testing

11. Screenshots or Demo

Include input screen, output result, dashboard

12. Known Issues

Accuracy depends on dataset, may misclassify unclear tickets

13. Future Enhancements

Use deep learning models, multilingual support, chatbot integration

Auto Ticket Classification

Project Documentation

Team ID: SWTID-2026-5313

1. Introduction

Project Title: Auto Ticket Classification System

Team ID: SWTID-2026-5313

Team Members:

Team Member
R Priyadharshini
K Pradeeba
Keerthika
Ponselvi

2. Project Overview

Purpose

The Auto Ticket Classification System is a web-based application designed to automatically classify support tickets based on the content and information provided by users. The system helps organizations organize large numbers of support tickets by automatically assigning them to appropriate categories, reducing manual classification effort and helping support teams process incoming tickets more efficiently.

Key Features

• User registration and login

• Creation of support tickets

• Automatic ticket classification

• Classification based on ticket description/content

• Category assignment

• Ticket status tracking

• Ticket management dashboard

• Search and filtering of tickets

• Admin/support-team management

• Ticket history and details

• REST API for ticket operations

• MongoDB database for storing ticket information

3. Architecture

Frontend – React

The frontend is developed using React.js. It provides login and registration pages, ticket creation, ticket lists, ticket details, classification results, an admin/support dashboard, search and filtering, and ticket status management. React communicates with the backend through REST APIs.

Backend – Node.js and Express.js

The backend is developed using Node.js with Express.js. It handles user authentication, ticket creation and management, classification requests, category assignment, REST APIs, MongoDB communication, validation, and authorization.

Database – MongoDB

MongoDB is used to store application data.

• Users: User ID, Name, Email, Password, Role

• Tickets: Ticket ID, User ID, Subject, Description, Category, Priority, Status, Created date, Updated date

• Classification Records: Ticket ID, Predicted category, Classification result, Classification date

4. Setup Instructions

Prerequisites

• Node.js

• npm

• MongoDB

• Git

• A modern web browser

• Code editor such as Visual Studio Code

Installation

```
git clone <repository-url>
cd auto-ticket-classification

cd client
npm install

cd ../server
npm install
```

Environment Variables

Create a .env file inside the server directory:

```
PORT=5000
MONGO_URI=<mongodb-connection-string>
JWT_SECRET=<your-secret-key>
```

5. Folder Structure

```
auto-ticket-classification/
■
■■■ client/
■   ■■■ public/
■   ■■■ src/
■       ■■■ components/
■       ■■■ pages/
■       ■■■ services/
```

```
■       ■■■ context/
■       ■■■ App.js
■       ■■■ index.js
■
■■■ server/
■   ■■■ controllers/
■   ■■■ models/
■   ■■■ routes/
■   ■■■ middleware/
■   ■■■ services/
■   ■■■ config/
■   ■■■ server.js
■   ■■■ .env
■
■■■ README.md
```

The client directory contains the React frontend. The server directory contains the Node.js/Express backend, database models, API routes, controllers, middleware, and classification-related services.

6. Running the Application

Start Backend

```
cd server
npm start
```

Example backend URL: http://localhost:5000

Start Frontend

```
cd client
npm start
```

Example frontend URL: http://localhost:3000

7. API Documentation

Authentication APIs

Register User — POST /api/auth/register

```
{
  "name": "User Name",
  "email": "user@example.com",
  "password": "password"
}
```

Login — POST /api/auth/login

```
{
  "email": "user@example.com",
  "password": "password"
}
```

Ticket APIs

Create Ticket — POST /api/tickets

Get Tickets — GET /api/tickets

Get Ticket by ID — GET /api/tickets/:id

Update Ticket — PUT /api/tickets/:id

Delete Ticket — DELETE /api/tickets/:id

Classification API — POST /api/tickets/:id/classify

```
{
  "ticketId": "12345",
  "category": "Account Support",
```

```
  "status": "Classified"
}
```

The exact endpoint names and response fields should be changed to match the implemented project APIs.

8. Authentication

The system can use authentication to ensure that only authorized users can access ticket-related functionality. The authentication system may include user registration, login, password protection, JWT-based authentication, role-based authorization, protected API routes, and session/token validation.

9. User Interface

• Login Page — Email, Password, Login button

• Registration Page — Name, Email, Password, Registration button

• Ticket Creation Page — Ticket subject, Ticket description, Submit button

• Ticket Dashboard — Ticket ID, Subject, Category, Priority, Status, Created date

• Ticket Details Page — Complete ticket description, automatically assigned category, status, ticket history

• Admin Dashboard — View tickets, filter tickets, manage categories, update ticket status, monitor classification results

Recommended screenshots include the login page, registration page, ticket creation page, ticket dashboard, submitted ticket, automatically classified ticket, ticket details page, admin dashboard, category filtering, and classification result.

10. Testing

The application should be tested to verify that ticket submission, classification, authentication, APIs, and database operations work correctly.

• User registration testing

• Login testing

• Ticket creation testing

• Ticket classification testing

• Category assignment testing

• Ticket update testing

• Ticket deletion testing

• API testing

• Authentication testing

• Database testing

• Frontend UI testing

Possible testing tools: Postman, Jest, React Testing Library, MongoDB testing, and browser-based manual testing.

11. Screenshots or Demo

Screenshots should demonstrate the major functionality of the Auto Ticket Classification System. Recommended screenshots: login page, registration page, ticket creation page, ticket dashboard, submitted ticket, automatically classified ticket, ticket details page, admin dashboard, category filtering, and classification result.

A demo link may also be provided if available.

12. Known Issues

Potential known issues should be documented based on the actual implementation.

• Classification may not always correctly identify ambiguous tickets.

• Classification accuracy depends on the quality and diversity of training data, if a machine-learning model is used.

• Incorrect or incomplete ticket descriptions may produce incorrect classifications.

• Internet/database connectivity may affect application performance.

• Newly introduced ticket categories may require updates to the classification system.

Only issues actually observed during development should be retained in the final documentation.

13. Future Enhancements

• Improved classification accuracy

• Support for additional ticket categories

• Priority prediction

• Automatic ticket assignment to support agents

• Email notifications

• Real-time ticket updates

• Advanced analytics dashboard

• Classification confidence scores

• Multilingual ticket classification

• Integration with external customer-support platforms

• Automated response suggestions

• Continuous improvement of the classification model

Project Summary

Item	Details
Team ID	SWTID-2026-5313
Project	Auto Ticket Classification
Team Members	R Priyadharshini; K Pradeeba; Keerthika; Ponselvi
Technology Stack	MERN
Frontend	React.js
Backend	Node.js + Express.js
Database	MongoDB
Core Functionality	Automatically classify incoming support tickets into relevant categories.

Project Report – Auto Ticket Classification

Team ID: SWTID-2026-5313

1. INTRODUCTION

Auto Ticket Classification is a system that automatically categorizes customer support tickets using AI and Machine Learning.

1.2 Purpose

To reduce manual effort, improve response time, and ensure accurate classification.

2. IDEATION PHASE

Manual classification is time-consuming and error-prone.

3. REQUIREMENT ANALYSIS

Includes dataset, ML model, preprocessing tools.

4. PROJECT DESIGN

System predicts ticket category using trained model.

5. PROJECT PLANNING

Week-wise plan for development and testing.

6. TESTING

Accuracy, Precision, Recall evaluation.

7. RESULTS

Shows classified ticket outputs.

8. ADVANTAGES

Saves time, improves efficiency.

9. CONCLUSION

Automation improves ticket management.

Project Planning Phase

Project Planning Template (Product Backlog, Sprint Planning, Stories, Story points)

Date	1 October 2026
Team ID	SWTID-2026-5313
Project Name	Auto Ticket Classification
Maximum Marks	5 Marks

Product Backlog, Sprint Schedule, and Estimation (4 Marks)

Use the below template to create product backlog and sprint schedule

Sprint	Functional Requirement (Epic)	User Story Number	User Story / Task	Story Points	Priority	Team Members
Sprint-1	Ticket Intake	USN-1	As a user, I can submit a support ticket with subject and description.	2	High	
Sprint-1	Ticket Intake	USN-2	As a system, I can validate and store ticket details for classification.	2	High	
Sprint-2	Text Processing	USN-3	As a system, I can preprocess ticket text before classification.	3	High	
Sprint-2	Classification	USN-4	As a system, I can predict the appropriate ticket category.	4	High	
Sprint-3	Priority & Routing	USN-5	As a system, I can predict the priority of a support ticket.	2	High	
Sprint-3	Priority & Routing	USN-6	As a system, I can route a classified ticket to the appropriate support team.	3	High	
Sprint-4	Dashboard & Evaluation	USN-7	As an admin, I can view ticket category, priority, and routing results.	2	Medium	
Sprint-4	Dashboard & Evaluation	USN-8	As an admin, I can view classification performance metrics.	2	Medium	

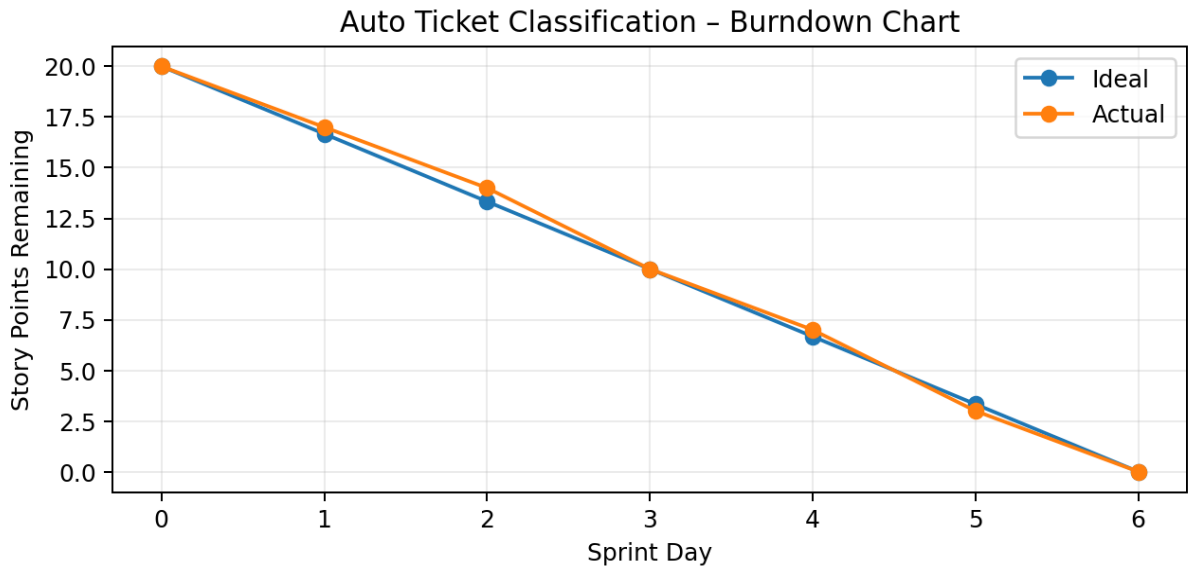
Project Tracker, Velocity & Burndown Chart: (4 Marks)

Sprint	Total Story Points	Duration	Sprint Start Date	Sprint End Date (Planned)	Story Points Completed (as on Planned End Date)	Sprint Release Date (Actual)
Sprint-1	4	6 Days	05 Oct 2026	10 Oct 2026	4	10 Oct 2026
Sprint-2	7	6 Days	12 Oct 2026	17 Oct 2026	7	17 Oct 2026
Sprint-3	5	6 Days	19 Oct 2026	24 Oct 2026	5	24 Oct 2026
Sprint-4	4	6 Days	26 Oct 2026	31 Oct 2026	4	31 Oct 2026

Velocity:

For the Auto Ticket Classification project, the planned work is 20 story points across four 6-day sprints. The average velocity is calculated as total completed story points divided by total sprint days.

Average Velocity (AV) = 20 story points ÷ 24 sprint-days = **0.83 story points/day**



Burndown Chart:

A burndown chart is a graphical representation of work left to do versus time. It is often used in Agile software development methodologies such as Scrum. For this project, the chart tracks the remaining story points as the Auto Ticket Classification features are completed.

Project-specific interpretation: The sprint starts with 20 story points. The remaining work reduces toward 0 as ticket intake, text processing, classification, priority/routing, and dashboard/evaluation tasks are completed.

Reference:

The Agile/Scrum planning structure in this document follows the supplied project planning template and its listed references.

https://www.atlassian.com/agile/project-management

https://www.atlassian.com/agile/tutorials/how-to-do-scrum-with-jira-software

https://www.atlassian.com/agile/tutorials/epics

https://www.atlassian.com/agile/tutorials/sprints

https://www.atlassian.com/agile/project-management/estimation

https://www.atlassian.com/agile/tutorials/burndown-charts